

15.071 — Deliverable 1

Cambridge Computers: Predicting Laptop Price

Problem 1 — linear regression predicting `Price` . **Problem 4** — logistic regression predicting whether a laptop is high-priced (`Price >= 500`).

Setup

```
In [1]: import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
from statsmodels.stats.anova import anova_lm
from scipy import stats

pd.set_option("display.width", 140)
pd.set_option("display.max_columns", 50)

train = pd.read_csv("laptop_train.csv")
test = pd.read_csv("laptop_test.csv")

print("train:", train.shape, "    test:", test.shape)
print("\nMissing values -- train:", train.isna().sum().sum(), "| test:", test.isna().sum().sum())
train.head()
```

train: (665, 9) test: (280, 9)

Missing values -- train: 0 | test: 0

```
Out[1]:
```

	InventoryID	Company	TypeName	GPU	Screen	Memory	Weight	Rating	Price
0	6	Asus	Gaming	Nvidia	17.3	16	2.90	1	2122.0
1	15	Asus	Gaming	Nvidia	17.3	16	2.73	3	2049.0
2	17	Asus	Gaming	Nvidia	15.6	16	2.50	4	1799.0
3	18	Asus	Gaming	Nvidia	17.3	16	4.00	10	998.0
4	38	Asus	Gaming	Nvidia	15.6	8	2.30	6	1649.0

```
In [2]: # Categorical levels (the first level alphabetically becomes the C() baseline)
for v in ["Company", "TypeName", "GPU"]:
    print(f"{v:10s} baseline = {sorted(train[v].unique())[0]:10s} | counts = {train[v].value_counts().sort_index().values}")

train[["Screen", "Memory", "Weight", "Rating", "Price"]].describe().round(3)
```

```

Company      baseline = Asus      | counts = {'Dell': 192, 'Lenovo': 186, 'H
P': 177, 'Asus': 110}
TypeName     baseline = Gaming   | counts = {'Notebook': 461, 'Ultrabook': 1
05, 'Gaming': 99}
GPU          baseline = AMD      | counts = {'Intel': 356, 'Nvidia': 194, 'A
MD': 115}

```

Out[2]:

	Screen	Memory	Weight	Rating	Price
count	665.000	665.000	665.000	665.000	665.000
mean	15.211	7.829	2.090	5.402	1026.673
std	1.180	3.772	0.607	2.854	573.910
min	12.500	4.000	0.910	1.000	224.000
25%	14.000	4.000	1.700	3.000	589.000
50%	15.600	8.000	2.060	5.000	899.000
75%	15.600	8.000	2.300	8.000	1280.000
max	17.300	16.000	4.600	10.000	3154.000

Problem 1 — Linear Regression

Problem 1(a)

Procedure followed.

1. Treat `Screen`, `Memory`, `Weight`, `Rating`, `Price` as numeric and `Company`, `TypeName`, `GPU` as categorical via `C()`.
2. Exclude `InventoryID`: it is an internal stocking identifier with no economic meaning, so including it risks fitting noise. The check below confirms it is also statistically insignificant.
3. Fit the full model with all remaining variables.
4. Backward elimination: test each variable for removal. For the categorical variables the correct test is a **partial F-test on the whole factor**, not the individual dummy p-values (dropping one level of a factor is not meaningful). All three factors and all four numeric variables are significant, so nothing is removed.

The model in step 4 is therefore the final model.

```

In [3]: # Step 2 -- is InventoryID worth keeping?
check = smf.ols("Price ~ C(Company) + C(TypeName) + C(GPU) + Screen + Memory
              data=train).fit()
print(f"InventoryID coefficient p-value = {check.pvalues['InventoryID']:.4f}")
print(f"Adj. R-squared with InventoryID = {check.rsquared_adj:.4f}")

```

InventoryID coefficient p-value = 0.4057 -> insignificant, drop it
Adj. R-squared with InventoryID = 0.6538

```
In [4]: # Final model
lin_formula = "Price ~ C(Company) + C(TypeName) + C(GPU) + Screen + Memory +
lin_model = smf.ols(lin_formula, data=train).fit()

print(f"Adj. R-squared without InventoryID = {lin_model.rsquared_adj:.4f} (
print(lin_model.summary())
```

Adj. R-squared without InventoryID = 0.6540 (higher -> confirms the drop)

OLS Regression Results

```
=====
==
Dep. Variable:          Price    R-squared:                0.6
60
Model:                  OLS      Adj. R-squared:            0.6
54
Method:                 Least Squares    F-statistic:              11
5.1
Date:                  Tue, 22 Sep 2026    Prob (F-statistic):        8.75e-1
45
Time:                  17:09:49    Log-Likelihood:            -480
9.1
No. Observations:      665    AIC:                      964
2.
Df Residuals:          653    BIC:                      969
6.
Df Model:              11
Covariance Type:       nonrobust
=====
```

```
=====
                                coef    std err          t      P>|t|
-----
[0.025    0.975]
-----
Intercept                1434.2466    288.621     4.969    0.000    86
7.510    2000.984
C(Company) [T.Dell]       86.7254    42.931     2.020    0.044
2.426    171.025
C(Company) [T.HP]        170.3954    43.502     3.917    0.000    8
4.974    255.816
C(Company) [T.Lenovo]     35.3279    42.129     0.839    0.402   -4
7.396    118.052
C(TypeName) [T.Notebook] -76.2954    58.721    -1.299    0.194   -19
1.600    39.009
C(TypeName) [T.Ultrabook] 416.3939    74.848     5.563    0.000    26
9.421    563.366
C(GPU) [T.Intel]         165.4297    39.663     4.171    0.000    8
7.547    243.312
C(GPU) [T.Nvidia]        218.1386    46.457     4.696    0.000    12
6.916    309.361
Screen                  -120.9323    21.801    -5.547    0.000   -16
3.741   -78.123
Memory                   87.3725     4.398    19.866    0.000    7
8.736    96.009
Weight                   271.0986    49.441     5.483    0.000    17
4.017    368.180
Rating                  -11.8206     4.613    -2.563    0.011   -2
0.878   -2.763
=====
```

```
=====
==
Omnibus:                78.159    Durbin-Watson:            1.7
98
Prob(Omnibus):          0.000    Jarque-Bera (JB):         152.7
```

```

65
Skew:                0.704    Prob(JB):                6.72e-
34
Kurtosis:            4.880    Cond. No.                40
6.
=====
==

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

In [5]: # Step 4 -- partial F-tests on each categorical factor as a whole
        factors = {
            "Company": ["C(Company) [T.Dell]", "C(Company) [T.HP]", "C(Company) [T.Ler
            "TypeName": ["C(TypeName) [T.Notebook]", "C(TypeName) [T.Ultrabook]"],
            "GPU":      ["C(GPU) [T.Intel]", "C(GPU) [T.Nvidia]"],
        }
        print("Partial F-tests (H0: all dummies for the factor are jointly zero)")
        for name, terms in factors.items():
            ft = lin_model.f_test([f"{t} = 0" for t in terms])
            print(f"    {name:9s} F = {float(ft.fvalue):7.3f}    p = {float(ft.pvalue):
        print("\nNumeric variable p-values")
        for v in ["Screen", "Memory", "Weight", "Rating"]:
            print(f"    {v:9s} p = {lin_model.pvalues[v]:.4f}    -> KEEP")

```

Partial F-tests (H0: all dummies for the factor are jointly zero)

Company	F =	6.736	p =	1.766e-04	-> KEEP
TypeName	F =	55.071	p =	7.913e-23	-> KEEP
GPU	F =	12.772	p =	3.623e-06	-> KEEP

Numeric variable p-values

Screen	p =	0.0000	-> KEEP
Memory	p =	0.0000	-> KEEP
Weight	p =	0.0000	-> KEEP
Rating	p =	0.0106	-> KEEP

Answer (a). The final model regresses `Price` on `C(Company) + C(TypeName) + C(GPU) + Screen + Memory + Weight + Rating` (summary output above). It has **$R^2 = 0.660$, adjusted $R^2 = 0.654$** on the training set, and the overall F-statistic is highly significant ($p \approx 8.8e-145$). Backward elimination removed only `InventoryID`; every other variable is significant at the 5% level, with the categorical variables judged by partial F-tests on the whole factor.

Problem 1(b)

Which independent variables are managerially sensible, and which warrant further investigation?

```
In [6]: coefs = pd.DataFrame({"coef": lin_model.params, "p_value": lin_model.pvalues
coefs["signif_5pct"] = np.where(coefs["p_value"] < 0.05, "YES", "no")
coefs["direction"] = np.where(coefs["coef"] > 0, "raises price", "lowers price")
print(coefs.round(4).to_string())
```

	coef	p_value	signif_5pct	direction
C(Company) [T.Dell]	86.7254	0.0438	YES	raises price
C(Company) [T.HP]	170.3954	0.0001	YES	raises price
C(Company) [T.Lenovo]	35.3279	0.4020	no	raises price
C(TypeName) [T.Notebook]	-76.2954	0.1943	no	lowers price
C(TypeName) [T.Ultrabook]	416.3939	0.0000	YES	raises price
C(GPU) [T.Intel]	165.4297	0.0000	YES	raises price
C(GPU) [T.Nvidia]	218.1386	0.0000	YES	raises price
Screen	-120.9323	0.0000	YES	lowers price
Memory	87.3725	0.0000	YES	raises price
Weight	271.0986	0.0000	YES	raises price
Rating	-11.8206	0.0106	YES	lowers price

Answer (b).

Managerially sensible:

- **Memory (+€87.37 per GB, $p < 0.001$)** — RAM is a direct, costly component upgrade. The single strongest predictor ($t \approx 19.9$).
- **GPU (Intel +€165, Nvidia +€218 vs AMD)** — discrete/premium graphics cost more than budget AMD integrated graphics.
- **TypeName = Ultrabook (+€416)** — ultrabooks are the premium thin-and-light segment.
- **Company (HP +€170, Dell +€87 vs Asus)** — consistent with brand positioning and the business orientation of HP/Dell lines.

Worthy of further investigation:

- **Screen (-€120.93 per inch, $p < 0.001$)** — counter-intuitive, since bigger panels cost more. Holding RAM, GPU, type and weight fixed, large screens mark cheap bulk notebooks while premium ultrabooks are small, so the coefficient is picking up segment rather than panel cost. Management should not conclude that shrinking a screen raises the price it can charge.
- **Rating (-€11.82 per point, $p = 0.011$)** — higher-rated laptops are *cheaper*. Plausibly value-for-money drives reviews (buyers of expensive machines are harder to please), but as a pricing input this is backwards and deserves scrutiny.
- **Weight (+€271 per kg, $p < 0.001$)** — ambiguous. It is sensible if weight proxies for more hardware (bigger battery, discrete GPU, cooling), but it contradicts the usual "thin-and-light commands a premium" intuition and is entangled with Screen and TypeName.

Problem 1(c)

Which manufacturer has the highest effect on price, and which the smallest?

```
In [7]: # Asus is the baseline level, so its effect is 0 by construction
company_effects = pd.Series({
    "Asus (baseline)": 0.0,
    "Dell": lin_model.params["C(Company) [T.Dell]"],
    "HP": lin_model.params["C(Company) [T.HP]"],
    "Lenovo": lin_model.params["C(Company) [T.Lenovo]"],
}).sort_values(ascending=False)

print("Effect on Price (Euros) relative to Asus, holding all else fixed:\n")
print(company_effects.round(2).to_string())
print(f"\nHighest effect: {company_effects.index[0]}")
print(f"Smallest effect: {company_effects.index[-1]}")
```

Effect on Price (Euros) relative to Asus, holding all else fixed:

HP	170.40
Dell	86.73
Lenovo	35.33
Asus (baseline)	0.00

Highest effect: HP

Smallest effect: Asus (baseline)

Answer (c). HP has the highest effect — an HP laptop is priced about **€170.40 more** than an otherwise identical Asus ($p < 0.001$). **Asus has the smallest effect:** it is the baseline level, and all three other manufacturers carry positive coefficients, so Asus is the cheapest brand holding all other characteristics fixed. Among the estimated coefficients the smallest is Lenovo (+€35.33), but it is not statistically distinguishable from Asus ($p = 0.402$).

Problem 1(d)

Out-of-sample R^2 on the test set.

```
In [8]: test_pred = lin_model.predict(test)

SSE = ((test["Price"] - test_pred) ** 2).sum() # errors of our model
SST = ((test["Price"] - train["Price"].mean()) ** 2).sum() # baseline = TRAIN MEAN
OSR2 = 1 - SSE / SST

print(f"Baseline (train mean price) = {train['Price'].mean():.4f} Euros")
print(f"SSE = {SSE:,.1f}")
print(f"SST = {SST:,.1f}")
print(f"OSR2 = {OSR2:.4f}")
```

Baseline (train mean price) = 1026.6727 Euros
SSE = 35,686,771.9
SST = 79,593,614.0
OSR2 = 0.5516

Answer (d). The out-of-sample R^2 is **0.5516**.

Interpretation: on laptops the model has never seen, it explains **55.16% of the variability in price** relative to the naive baseline that predicts every test laptop's price to be the average price in the training set (€1,026.67). Equivalently, the model's sum of squared prediction errors on the test set is 55.16% smaller than the baseline's. It is lower than the training R^2 of 0.660, which is the expected drop from in-sample fit to genuine out-of-sample performance.

Problem 1(e)

Probability that the predicted price of the given laptop exceeds €1,100.

Assumptions. The linear regression assumes the error term is normally distributed with mean 0 and constant variance, $\varepsilon \sim N(0, \sigma^2)$, independent across laptops. So for a laptop with characteristics x ,

$$\text{Price}|x \sim N(\hat{y}(x), \sigma^2), \quad P(\text{Price} > 1100) = 1 - \Phi\left(\frac{1100 - \hat{y}(x)}{\sigma}\right)$$

We estimate σ by the residual standard error of the fitted model, and (as is standard for this calculation) we ignore the extra uncertainty in the estimated coefficients themselves.

```
In [9]: new_laptop = pd.DataFrame({
    "InventoryID": [950],
    "Company":     ["Asus"],
    "TypeName":    ["Ultrabook"],
    "GPU":         ["Intel"],
    "Screen":      [15.6],
    "Memory":      [6],
    "Weight":      [3.0],
    "Rating":      [8],
})

y_hat = lin_model.predict(new_laptop).iloc[0]
sigma = np.sqrt(lin_model.scale)           # residual standard error
z      = (1100 - y_hat) / sigma
prob   = 1 - stats.norm.cdf(z)

print(f" Point prediction y_hat = {y_hat:.4f} Euros")
print(f" Residual std error sigma = {sigma:.4f} Euros    (df = {int(lin_model.df_resid)})")
print(f" z = (1100 - y_hat)/sigma = {z:.4f}")
print(f" P(Price > 1100)          = {prob:.4f}")

# robustness: t-distribution, and including coefficient-estimation uncertainty
prob_t = 1 - stats.t.cdf(z, df=lin_model.df_resid)
se_pred = np.sqrt(lin_model.get_prediction(new_laptop).se_mean[0] ** 2 + lin_model.scale)
prob_full = 1 - stats.norm.cdf((1100 - y_hat) / se_pred)
print(f"\n [check] using t({int(lin_model.df_resid)}) instead of normal : {prob_t:.4f}")
print(f" [check] including coefficient uncertainty (se = {se_pred:.2f}) : {prob_full:.4f}")
```



```
Point prediction y_hat = 1372.4919 Euros
Residual std error sigma = 337.5865 Euros    (df = 653)
z = (1100 - y_hat)/sigma = -0.8072
P(Price > 1100)          = 0.7902
```

```
[check] using t(653) instead of normal : 0.7901
```

```
[check] including coefficient uncertainty (se = 347.01) : 0.7838
```

Answer (e). The point prediction is **€1,372.49** with residual standard error $\sigma =$ **€337.59**, so

$$P(\text{Price} > 1100) = 1 - \Phi\left(\frac{1100 - 1372.49}{337.59}\right) = 1 - \Phi(-0.807) \approx \mathbf{0.790}$$

There is roughly a **79% probability** that this laptop's price exceeds €1,100. The calculation relies on the normal-errors assumption stated above; using a t-distribution gives the same 0.790, and additionally accounting for uncertainty in the estimated coefficients gives 0.784, so the answer is not sensitive to those choices. (Caveat: the residuals are right-skewed — the summary reports skew 0.70 and a significant Jarque-Bera — so the normality assumption is only approximate.)

Problem 1(f)

Add an interaction between `Memory` and `GPU` .

```
In [10]: lin_f = smf.ols(lin_formula + " + Memory:C(GPU)", data=train).fit()
print(lin_f.summary().tables[1])

print("\nJoint test of the two interaction terms (nested F-test):")
print(anova_lm(lin_model, lin_f).round(4).to_string())
```

[0.025 0.975]		coef	std err	t	P> t	
Intercept		1635.8257	293.069	5.582	0.000	106
0.351	2211.301					
C(Company) [T.Dell]		82.8037	42.702	1.939	0.053	–
1.046	166.653					
C(Company) [T.HP]		160.9213	43.296	3.717	0.000	7
5.905	245.938					
C(Company) [T.Lenovo]		33.3374	41.937	0.795	0.427	–4
9.010	115.685					
C(TypeName) [T.Notebook]		–72.5664	59.035	–1.229	0.219	–18
8.488	43.356					
C(TypeName) [T.Ultrabook]		395.3125	76.346	5.178	0.000	24
5.399	545.226					
C(GPU) [T.Intel]		–92.4312	88.996	–1.039	0.299	–26
7.185	82.323					
C(GPU) [T.Nvidia]		–29.5910	99.965	–0.296	0.767	–22
5.883	166.701					
Screen		–120.3982	21.728	–5.541	0.000	–16
3.064	–77.733					
Memory		58.3904	9.763	5.981	0.000	3
9.220	77.561					
Memory:C(GPU) [T.Intel]		37.2253	11.721	3.176	0.002	1
4.209	60.242					
Memory:C(GPU) [T.Nvidia]		32.9786	11.464	2.877	0.004	1
0.467	55.490					
Weight		271.2845	50.406	5.382	0.000	17
2.307	370.262					
Rating		–11.9920	4.582	–2.617	0.009	–2
0.990	–2.995					

Joint test of the two interaction terms (nested F-test):

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	653.0	7.441893e+07	0.0	NaN	NaN	NaN
1	651.0	7.317176e+07	2.0	1.247174e+06	5.548	0.0041

```
In [11]: # Implied effect of one extra GB of RAM, by GPU
print("Marginal effect of +1 GB of Memory on Price (Euros):")
print(f"  AMD      (baseline) : {lin_f.params['Memory']:8.2f}")
print(f"  Intel      : {lin_f.params['Memory'] + lin_f.params['Memory']}")
print(f"  Nvidia     : {lin_f.params['Memory'] + lin_f.params['Memory']}")

def osr2(model):
    pred = model.predict(test)
    return 1 - ((test["Price"] - pred) ** 2).sum() / ((test["Price"] - train

print(f"\n          R2      adjR2      OSR2")
print(f" part (a) {lin_model.rsquared:.4f} {lin_model.rsquared_adj:.4f}
print(f" part (f) {lin_f.rsquared:.4f} {lin_f.rsquared_adj:.4f} {osr2(li
```

Marginal effect of +1 GB of Memory on Price (Euros):

AMD (baseline)	:	58.39
Intel	:	95.62
Nvidia	:	91.37

	R2	adjR2	OSR2
part (a)	0.6597	0.6540	0.5516
part (f)	0.6654	0.6587	0.5507

Answer (f).

Interpretation of the interaction coefficients. `Memory:C(GPU)[T.Intel] = 37.23`

means that the price impact of **one extra GB of RAM is €37.23 larger on an Intel-GPU laptop than on an AMD-GPU laptop** (the baseline). Likewise `Memory:C(GPU)`

`[T.Nvidia] = 32.98` says the per-GB effect is €32.98 larger for Nvidia than for AMD.

So RAM is worth about €58 per GB on an AMD machine but €96 per GB on an Intel machine and €91 per GB on an Nvidia machine.

How the model differs from (a). The interaction is jointly significant ($F = 5.55$, $p = 0.004$) and R^2 rises slightly from 0.660 to 0.665, and the `Memory` coefficient now means the AMD-only slope (58.39) rather than a single common slope (87.37), while the `GPU` main effects become insignificant because they now describe the GPU gap at the meaningless point `Memory = 0`. The gain is in-sample only — out-of-sample R^2 actually falls slightly ($0.5516 \rightarrow 0.5507$), so the added complexity does not improve prediction.

Problem 1(g)

Add an interaction between `GPU` and `Company`.

```
In [12]: lin_g = smf.ols(lin_formula + " + C(GPU):C(Company)", data=train).fit()
print(lin_g.summary().tables[1])

print("\nJoint test of the six interaction terms (nested F-test):")
print(anova_lm(lin_model, lin_g).round(4).to_string())
```

P> t			coef	std err	t
[0.025 0.975]					
Intercept			1710.5748	309.677	5.524
0.000	1102.482	2318.667			
C(Company) [T.Dell]			-286.9438	135.710	-2.114
0.035	-553.428	-20.459			
C(Company) [T.HP]			-170.4844	140.795	-1.211
0.226	-446.955	105.986			
C(Company) [T.Lenovo]			-322.1718	148.715	-2.166
0.031	-614.194	-30.150			
C(TypeName) [T.Notebook]			-15.8465	59.018	-0.269
0.788	-131.736	100.043			
C(TypeName) [T.Ultrabook]			488.9254	74.667	6.548
0.000	342.307	635.543			
C(GPU) [T.Intel]			-279.7797	138.648	-2.018
0.044	-552.035	-7.524			
C(GPU) [T.Nvidia]			-45.8472	132.522	-0.346
0.729	-306.073	214.379			
C(GPU) [T.Intel]:C(Company) [T.Dell]			423.9139	148.829	2.848
0.005	131.667	716.161			
C(GPU) [T.Nvidia]:C(Company) [T.Dell]			477.2737	150.926	3.162
0.002	180.909	773.638			
C(GPU) [T.Intel]:C(Company) [T.HP]			464.5333	152.415	3.048
0.002	165.245	763.822			
C(GPU) [T.Nvidia]:C(Company) [T.HP]			150.2330	159.453	0.942
0.346	-162.875	463.341			
C(GPU) [T.Intel]:C(Company) [T.Lenovo]			508.6057	160.102	3.177
0.002	194.223	822.988			
C(GPU) [T.Nvidia]:C(Company) [T.Lenovo]			229.8932	159.225	1.444
0.149	-82.767	542.553			
Screen			-118.4726	21.867	-5.418
0.000	-161.412	-75.533			
Memory			86.1194	4.327	19.901
0.000	77.622	94.617			
Weight			259.0412	49.048	5.281
0.000	162.729	355.353			
Rating			-11.5912	4.532	-2.558
0.011	-20.491	-2.692			

Joint test of the six interaction terms (nested F-test):

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	653.0	7.441893e+07	0.0	NaN	NaN	NaN
1	647.0	7.076556e+07	6.0	3.653368e+06	5.567	0.0

```
In [13]: # Implied Intel-vs-AMD and Nvidia-vs-AMD premium, by manufacturer
rows = []
for comp in ["Asus", "Dell", "HP", "Lenovo"]:
    row = {"Company": comp}
    for gpu in ["Intel", "Nvidia"]:
        eff = lin_g.params[f"C(GPU) [T.{gpu}]"]
```

```

if comp != "Asus":
    eff += lin_g.params.get(f"C(GPU) [T.{gpu}]:C(Company) [T.{comp}]",
    row[f"{gpu} vs AMD"] = eff
rows.append(row)
print("Price premium of each GPU relative to AMD, within manufacturer (Euros)
print(pd.DataFrame(rows).set_index("Company").round(2).to_string())

print(f"\n          R2      adjR2      OSR2")
print(f" part (a) {lin_model.rsquared:.4f} {lin_model.rsquared_adj:.4f}
print(f" part (g) {lin_g.rsquared:.4f} {lin_g.rsquared_adj:.4f} {osr2(li

```

Price premium of each GPU relative to AMD, within manufacturer (Euros):

	Intel vs AMD	Nvidia vs AMD
Company		
Asus	-279.78	-45.85
Dell	144.13	431.43
HP	184.75	104.39
Lenovo	228.83	184.05

	R2	adjR2	OSR2
part (a)	0.6597	0.6540	0.5516
part (g)	0.6764	0.6679	0.5641

Answer (g).

Interpretation of the interaction coefficients. Each term is a difference-in-differences between two categorical variables. For example `C(GPU) [T.Intel]:C(Company) [T.Dell] = 423.91` means the **Intel-versus-AMD price premium is €423.91 larger for Dell laptops than it is for Asus laptops** (the baseline company). In other words, the value of the GPU brand is allowed to depend on who makes the laptop: as the table shows, moving from AMD to Intel is worth -€280 within Asus but about +€144 within Dell and +€185 within HP.

How the model differs from (a). The six interaction terms are jointly significant ($F = 5.57$, $p < 0.001$) and R^2 rises from 0.660 to 0.676, and unlike part (f) this model also improves out-of-sample (OSR^2 0.5516 $\rightarrow$ 0.5641), so allowing the GPU premium to vary by manufacturer is a genuine improvement. The `Company` and `GPU` main effects flip to negative because they now describe only the AMD laptops and the Asus laptops respectively, not overall brand effects.

Problem 4 — Logistic Regression

Management now wants to classify a laptop as **high-priced** (`Price >= 500`) or not.

Problem 4 setup: create the binary outcome `high`

`high = 1` if `Price >= 500` Euros, `high = 0` otherwise, in **both** train and test.

```
In [14]: train["high"] = (train["Price"] >= 500).astype(int)
test["high"] = (test["Price"] >= 500).astype(int)

print("Train high counts:\n", train["high"].value_counts().sort_index().to_s
print("\nTest high counts:\n", test["high"].value_counts().sort_index().to_s
print(f"\nProportion high -- train: {train['high'].mean():.4f} | test: {test
```

Train high counts:

high	
0	115
1	550

Test high counts:

high	
0	53
1	227

Proportion high -- train: 0.8271 | test: 0.8107

Problem 4(a)

Logistic regression predicting `high` from all independent variables in Table 1. **No variable selection is performed.**

```
In [15]: log_formula = "high ~ C(Company) + C(TypeName) + C(GPU) + Screen + Memory +
log_model = smf.logit(log_formula, data=train).fit()
print(log_model.summary())
```

Warning: Maximum number of iterations has been exceeded.

Current function value: 0.251696

Iterations: 35

Logit Regression Results

```
=====
==
Dep. Variable:          high    No. Observations:          6
65
Model:                  Logit    Df Residuals:          6
53
Method:                 MLE     Df Model:
11
Date:                   Tue, 22 Sep 2026    Pseudo R-squ.:          0.45
34
Time:                   17:09:49    Log-Likelihood:         -167.
38
converged:              False    LL-Null:                -306.
24
Covariance Type:        nonrobust    LLR p-value:            4.281e-
53
=====
```

```
=====
=====
                                coef    std err          z      P>|z|
-----
[0.025    0.975]
-----
Intercept                    29.4861    5959.094     0.005     0.996    -1.17
e+04    1.17e+04
C(Company) [T.Dell]          1.2456     0.501     2.486     0.013
0.263    2.228
C(Company) [T.HP]            1.4910     0.465     3.206     0.001
0.579    2.403
C(Company) [T.Lenovo]        -0.0525     0.445    -0.118     0.906     -
0.925    0.820
C(TypeName) [T.Notebook]    -17.6031    5959.093    -0.003     0.998    -1.17
e+04    1.17e+04
C(TypeName) [T.Ultrabook]   -16.1946    5959.093    -0.003     0.998    -1.17
e+04    1.17e+04
C(GPU) [T.Intel]             -0.1053     0.354    -0.298     0.766     -
0.799    0.589
C(GPU) [T.Nvidia]           2.0148     0.605     3.329     0.001
0.829    3.201
Screen                       -1.1954     0.311    -3.845     0.000     -
1.805    -0.586
Memory                       0.7421     0.093     8.019     0.000
0.561    0.923
Weight                       1.5361     0.903     1.701     0.089     -
0.234    3.306
Rating                       -0.0904     0.049    -1.838     0.066     -
0.187    0.006
=====
=====
```

Possibly complete quasi-separation: A fraction 0.20 of observations can be perfectly predicted. This might indicate that there is complete quasi-separation. In this case some parameters will not be identified.

```
/Users/chrislowzx/miniconda3/lib/python3.13/site-packages/statsmodels/base/m
odel.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to c
onverge. Check mle_retvals
warnings.warn("Maximum Likelihood optimization failed to "
```

Answer (a). The fitted logistic regression model using all independent variables is shown above. The model produces a convergence warning due to all Gaming laptops in the training data are high-priced. So some TypeName coefficient estimates should be interpreted with caution.

Problem 4(b)

Which variables are significant in predicting the probability of a high price?

```
In [17]: sig = pd.DataFrame({"coef": log_model.params, "p_value": log_model.pvalues})
sig["signif_5pct"] = np.where(sig["p_value"] < 0.05, "YES", "no")
print(sig.round(4).to_string())

print("\nSignificant at 5%:", [v for v in sig.index[sig["p_value"] < 0.05] if
```

	coef	p_value	signif_5pct
Intercept	29.4861	0.9961	no
C(Company) [T.Dell]	1.2456	0.0129	YES
C(Company) [T.HP]	1.4910	0.0013	YES
C(Company) [T.Lenovo]	-0.0525	0.9061	no
C(TypeName) [T.Notebook]	-17.6031	0.9976	no
C(TypeName) [T.Ultrabook]	-16.1946	0.9978	no
C(GPU) [T.Intel]	-0.1053	0.7661	no
C(GPU) [T.Nvidia]	2.0148	0.0009	YES
Screen	-1.1954	0.0001	YES
Memory	0.7421	0.0000	YES
Weight	1.5361	0.0889	no
Rating	-0.0904	0.0660	no

```
Significant at 5%: ['C(Company) [T.Dell]', 'C(Company) [T.HP]', 'C(GPU) [T.Nvid
ia]', 'Screen', 'Memory']
```

Answer (b). At the 5% level the significant predictors are **Memory** ($p < 0.001$), **Screen** ($p < 0.001$), **GPU = Nvidia** ($p = 0.001$), and **Company = HP** ($p = 0.001$) and **Company = Dell** ($p = 0.013$) relative to the Asus baseline. **Weight** ($p = 0.089$) and **Rating** ($p = 0.066$) are significant only at the 10% level, and **GPU = Intel** and **Company = Lenovo** are not significant.

Note that the **TypeName** p-values (≈ 0.998) are meaningless here because of the separation described in part (a).

Problem 4(c)

Are the significant variables the same as in the Problem 1 linear model?


```
In [18]: compare = pd.DataFrame({
    "linear_coef": lin_model.params,
    "linear_p": lin_model.pvalues,
    "logit_coef": log_model.params,
    "logit_p": log_model.pvalues,
})
compare["linear_sig_5pct"] = np.where(compare["linear_p"] < 0.05, "YES", "no")
compare["logit_sig_5pct"] = np.where(compare["logit_p"] < 0.05, "YES", "no")
compare["same_significance"] = np.where(
    compare["linear_sig_5pct"] == compare["logit_sig_5pct"], "same", "DIFFERENT")
print(compare.round(4).to_string())
```

		linear_coef	linear_p	logit_coef	logit_p	linear_
sig_5pct	logit_sig_5pct	same_significance				
Intercept		1434.2466	0.0000	29.4861	0.9961	
YES	no	DIFFERENT				
C(Company) [T.Dell]		86.7254	0.0438	1.2456	0.0129	
YES	YES	same				
C(Company) [T.HP]		170.3954	0.0001	1.4910	0.0013	
YES	YES	same				
C(Company) [T.Lenovo]		35.3279	0.4020	-0.0525	0.9061	
no	no	same				
C(TypeName) [T.Notebook]		-76.2954	0.1943	-17.6031	0.9976	
no	no	same				
C(TypeName) [T.Ultrabook]		416.3939	0.0000	-16.1946	0.9978	
YES	no	DIFFERENT				
C(GPU) [T.Intel]		165.4297	0.0000	-0.1053	0.7661	
YES	no	DIFFERENT				
C(GPU) [T.Nvidia]		218.1386	0.0000	2.0148	0.0009	
YES	YES	same				
Screen		-120.9323	0.0000	-1.1954	0.0001	
YES	YES	same				
Memory		87.3725	0.0000	0.7421	0.0000	
YES	YES	same				
Weight		271.0986	0.0000	1.5361	0.0889	
YES	no	DIFFERENT				
Rating		-11.8206	0.0106	-0.0904	0.0660	
YES	no	DIFFERENT				

Answer (c). The significant predictors are not identical between the two models.

Memory, Screen, GPU = Nvidia, Company = Dell, and **Company = HP** are significant in both, while **GPU = Intel, Weight, Rating,** and **TypeName = Ultrabook** are significant only in the linear model.

Problem 4(d)

For each significant variable, does an increase in its value raise or lower the probability of being high-priced?

```
In [19]: signif = compare.loc[(compare["logit_p"] < 0.05) & (compare.index != "Intercept")]
signif["odds_ratio"] = np.exp(signif["logit_coef"])
```

```
signif["direction"] = np.where(signif["logit_coef"] > 0, "INCREASES P(high)", "DECREASES P(high)")
print(signif[["logit_coef", "odds_ratio", "logit_p", "direction"]].round(4).)
```

	logit_coef	odds_ratio	logit_p	direction
C(Company) [T.Dell]	1.2456	3.4750	0.0129	INCREASES P(high)
C(Company) [T.HP]	1.4910	4.4416	0.0013	INCREASES P(high)
C(GPU) [T.Nvidia]	2.0148	7.4991	0.0009	INCREASES P(high)
Screen	-1.1954	0.3026	0.0001	DECREASES P(high)
Memory	0.7421	2.1003	0.0000	INCREASES P(high)

Answer (d).

Among the significant predictors, higher **Memory** increases the probability of being high-priced (coefficient = 0.742, odds ratio = 2.10), which makes sense since additional RAM is generally associated with more expensive laptops.

Nvidia GPUs (coefficient = 2.015, OR = 7.50), **HP** (1.491, OR = 4.44), and **Dell** (1.246, OR = 3.48) also increase the probability relative to their baseline categories, AMD and Asus. This makes sense, as Nvidia GPUs are often connected to higher-performance laptops, while the positive HP and Dell effects suggest that laptops from these manufacturers are more likely to be high-priced than similar Asus laptops in this dataset.

In contrast, a larger **Screen** decreases the probability (coefficient = -1.195, OR = 0.30). This is somewhat counterintuitive since larger screens might be expected to increase price, but the data suggests the opposite after controlling for the other laptop characteristics.

Problem 4(e)

For which variables do the linear and logistic coefficients share a sign, and for which do they differ?

```
In [20]: signs = compare[["linear_coef", "logit_coef", "linear_p", "logit_p"]].copy()
signs["linear_sign"] = np.where(signs["linear_coef"] > 0, "+", "-")
signs["logit_sign"] = np.where(signs["logit_coef"] > 0, "+", "-")
signs["agreement"] = np.where(signs["linear_sign"] == signs["logit_sign"], "yes", "no")
print(signs.round(4).to_string())
```

			linear_coef	logit_coef	linear_p	logit_p	linear_
sign	logit_sign	agreement					
Intercept			1434.2466	29.4861	0.0000	0.9961	
+	+	SAME					
C(Company)	[T.Dell]		86.7254	1.2456	0.0438	0.0129	
+	+	SAME					
C(Company)	[T.HP]		170.3954	1.4910	0.0001	0.0013	
+	+	SAME					
C(Company)	[T.Lenovo]		35.3279	-0.0525	0.4020	0.9061	
+	-	DIFFERENT					
C(TypeName)	[T.Notebook]		-76.2954	-17.6031	0.1943	0.9976	
-	-	SAME					
C(TypeName)	[T.Ultrabook]		416.3939	-16.1946	0.0000	0.9978	
+	-	DIFFERENT					
C(GPU)	[T.Intel]		165.4297	-0.1053	0.0000	0.7661	
+	-	DIFFERENT					
C(GPU)	[T.Nvidia]		218.1386	2.0148	0.0000	0.0009	
+	+	SAME					
Screen			-120.9323	-1.1954	0.0000	0.0001	
-	-	SAME					
Memory			87.3725	0.7421	0.0000	0.0000	
+	+	SAME					
Weight			271.0986	1.5361	0.0000	0.0889	
+	+	SAME					
Rating			-11.8206	-0.0904	0.0106	0.0660	
-	-	SAME					

Answer (e).

The coefficients have the **same sign** for **Company = Dell (+)**, **Company = HP (+)**, **TypeName = Notebook (-)**, **GPU = Nvidia (+)**, **Screen (-)**, **Memory (+)**, **Weight (+)**, and **Rating (-)**.

The signs are **different** for **Company = Lenovo** (+ in linear, - in logistic), **GPU = Intel** (+ in linear, - in logistic), and **TypeName = Ultrabook** (+ in linear, - in logistic). However, again, the **TypeName** logistic coefficient should be interpreted cautiously because of the quasi-complete separation identified in part (a).

Problem 4(f)

Probability that a Lenovo Ultrabook (Intel GPU), **Screen = 8**, **Memory = 8**, **Weight = 4.2**, **Rating = 7** is high-priced.

$$P(\text{high} = 1 \mid x) = \frac{1}{1 + e^{-z}}, \quad z = \beta_0 + \beta_{\text{Lenovo}} + \beta_{\text{Ultrabook}} + \beta_{\text{Intel}} + \beta_{\text{Screen}}(8) + \beta_{\text{Memory}}(8) + \beta_{\text{Weight}}(4.2) + \beta_{\text{Rating}}(7)$$

The Dell/HP, Notebook and Nvidia indicators are all 0 for this laptop, so they drop out.

```
In [21]: new_laptop_4 = pd.DataFrame({
        "InventoryID": [4096],
```

```

    "Company":      ["Lenovo"],
    "TypeName":     ["Ultrabook"],
    "GPU":          ["Intel"],
    "Screen":       [8.0],
    "Memory":       [8],
    "Weight":       [4.2],
    "Rating":       [7],
})

b = log_model.params
terms = {
    "Intercept":      b["Intercept"],
    "Company=Lenovo": b["C(Company) [T.Lenovo]"],
    "TypeName=Ultrabook": b["C(TypeName) [T.Ultrabook]"],
    "GPU=Intel":      b["C(GPU) [T.Intel]"],
    "Screen x 8.0":   b["Screen"] * 8.0,
    "Memory x 8":     b["Memory"] * 8,
    "Weight x 4.2":   b["Weight"] * 4.2,
    "Rating x 7":     b["Rating"] * 7,
}
for k, v in terms.items():
    print(f" {k:<20} = {v:>12.4f}")

z_val      = sum(terms.values())
p_manual   = 1 / (1 + np.exp(-z_val))
p_smf      = log_model.predict(new_laptop_4).iloc[0]

print(f"\n z (log-odds)           = {z_val:.4f}")
print(f" P(high=1) manual          = {p_manual:.8f}")
print(f" P(high=1) via .predict       = {p_smf:.8f}")
print(f" 1 - P(high=1)                = {1 - p_manual:.3e}")

```

```

Intercept           =      29.4861
Company=Lenovo      =      -0.0525
TypeName=Ultrabook  =     -16.1946
GPU=Intel           =     -0.1053
Screen x 8.0        =     -9.5635
Memory x 8          =      5.9365
Weight x 4.2        =      6.4516
Rating x 7          =     -0.6325

```

```

z (log-odds)        = 15.3257
P(high=1) manual     = 0.99999978
P(high=1) via .predict = 0.99999978
1 - P(high=1)        = 2.209e-07

```

Answer (f). Substituting the fitted coefficients gives a log-odds of $z \approx 15.33$, so

$$P(\text{high} = 1) = \frac{1}{1 + e^{-15.33}} \approx 0.99999978 \approx 1.00$$

The model predicts this laptop is essentially **certain to be high-priced**. Two caveats:

Screen = 8 inches lies far outside the training range (12.5–17.3 in), so this is an extrapolation; and the intercept and TypeName coefficients are individually unidentified

because of the separation, although their *sum* (which is what enters the prediction) is well determined.

Answer (f).

For this laptop, the fitted log-odds are

$$z = 29.4861 - 0.0525 - 16.1946 - 0.1053 - 1.1954(8) + 0.7421(8) + 1.5361(4.2) - (\\ \approx 15.33.$$

Therefore,

$$P(\text{high} = 1) = \frac{1}{1 + e^{-15.33}} \approx 0.99999978.$$

Thus, the fitted model predicts an approximately **100% probability** that this laptop is high-priced. This prediction should be interpreted cautiously because the logistic regression had the quasi-complete separation issue identified in part (a).

Problem 4(g)

Apply the model to the test set with a 0.5 probability cutoff and compute accuracy.

```
In [22]: test_probs = log_model.predict(test)
test_class = (test_probs >= 0.5).astype(int)

conf = pd.crosstab(test["high"], test_class, rownames=["Actual"], colnames=["Predicted"])
print("Confusion matrix (test set):")
print(conf.to_string(), "\n")

n_correct = (test_class == test["high"]).sum()
accuracy = n_correct / len(test)
baseline = test["high"].mean() # naive rule: always predict "high"

print(f"Test accuracy (cutoff 0.5) = {accuracy:.4f} ({n_correct} / {len(test)})")
print(f"Baseline accuracy = {baseline:.4f}")
```

Confusion matrix (test set):

Predicted	0	1
Actual		
0	27	26
1	15	212

Test accuracy (cutoff 0.5) = 0.8536 (239 / 280)

Baseline accuracy = 0.8107

Answer (g).

Using a probability cutoff of 0.5, the model correctly classifies **239 out of 280** laptops in the test set. Thus, the test accuracy is

$$\text{Accuracy} = \frac{239}{280} = 0.8536 \approx 85.4\%.$$

This is slightly better than the **81.1% naive baseline** from always predicting a laptop as high-priced.